

Хранение секретов в коде

Описание проблемы

Закодированные учетные данные создают значительную опасность, позволяющую злоумышленнику обойти системы аутентификации. В подобных случаях, сложно обнаружить взлом и источник утечки.

Виды конф данных на которые сразу обращаем внимание:

- Учетные записи: логин-пароль, токены, sid, ключи доступа к внешним сервисам, oauth
- Аккаунты приложений: конфигурации приложений, app_secret, secret_key
- Шифрование: ключи шифрования частично или полностью

Проблема усугубляется общедоступностью исходников внутри компании.

Примеры небезопасного кода

Все примеры взяты из практики, секреты изменены.

Хранение сертификатов в коде:

```
1 serial_val =  
  "MIH6BgkqhkiG9w0BwOggewwgekCAQAxgZijgZUCAQCgPQYJKoZIhvcNAQUMMDAEIMRtoi8  
  zFgTx29Vz3ldrYYJlH8pKmlmLVRuiaBHW0toUAgID6KAIBgYqhQMCAGowCAYGKoUDAgIVBE  
  cBIAAAHmYAA588idTaKYfRKbuQemZHPxfqnovMilQyxevACLE1/7Ku5MwCQYHKoUDAgIfAT  
  BJBgkqhkiG9w0BBwEwHQYGGKoUDAgIVBMECObP3kzfJF5kBgcqhQMCAh8BgB2DdqvtYmitf  
  5UECD77FTZpCg=="  
2 serial_data_file = RpcFile()  
3 serial_data_file.SetData(b64decode(serial_val.encode('utf-8')))  
4 decr_val = EncryptedMessage.DecryptByPassword(serial_data_file,  
  spec_feature_name)  
5 ....
```

Хранение ключей доступа в коде метода ogx:

```
1 ...  
2 from collections import defaultdict  
3 secret = 'c71b1e56-7568-4b68-b098-8ece21234b27'  
4 if key == secret:  
5 ...
```

Код теста содержит тестовые креды и sid:

```
1 ...  
2 from unit_tests.create_ad_obj import ad_tools  
3 # test unittest/unitest1234  
4 session.set_session('009a3a8f-00c22d95-00ba-0j09b599874a5728',  
  '894d90o0-6445-4085-a3G0-f6b16ad4frF4')  
5 TEST_DATA = {  
6 ...
```

Ini файл

```
1 [Базовая конфигурация]  
2 БазаДанных=postgresql: host='test-test-db.unix.tensor.ru' port='5432'  
  dbname='css' user='Postgres' password='Postgres123'  
3 ...
```

Кроме того, секрет нельзя передавать в качестве параметра командной строки, так как он остаётся в истории команд и будет отображаться в списке выполняемых процессов.

Недопустимо использование тестовых учёток на боевых стендах, начиная с fix.

Последствия эксплуатации

Последствия утечки могут быть разнообразными. Утечка учетных данных может привести к получению доступа к аккаунтам и упрощению атак на инфраструктуру компании. Например:

1. Доступ к базам данных, серверам, сетевым хранилищам может использоваться для порчи или подмены данных клиентов.
2. Утечка боевых или fix логина-пароля, sid, тикета или токена от боевых сайтов позволит:
 1. Получить доступ к внутренним документам компании и переписке
 2. Получить доступ к аккаунтам клиентов
 3. Удаленный доступ к рабочим ПК через удаленный помощник
 4. Нанести репутационный ущерб компании
3. Диверсии на наших или сторонних сервисах через аккаунт компании.
4. Дискредитации клиентов или нашей компании.

Как исправить?

Важнейшим шагом исправления утечки является смена скомпрометированного секрета!

Безопасные способы использования секретов:

- Наиболее предпочтительный способ - использовать сервис хранения секретов Omerta. Подробнее о его использовании можно почитать в описании [API](#) сервиса.
- Хранить данные в [параметрах конфигурации](#) облака с типом "Пароль".
- Использовать [Vault](#).
- Задавать секреты в [переменных среды](#).
- Локальный ini файл может содержать секреты при условии контроля доступа к нему.

Сейчас не всегда есть возможность убирать из кода учетки для автотестирования. Что делать в таком случае - рассказано в [новости](#). Это временная мера и не должна использоваться, как полноценная замена безопасному хранению секретов.